

indicators

June 19, 2025

1 Indikatoren für XAU berechnen und exportieren

```
[13]: import pandas as pd
import os
from ta.momentum import RSIIndicator
from ta.trend import MACD, SMAIndicator

# 1. Daten laden
df = pd.read_csv("../dataset/output/xau_clean.csv", parse_dates=["date_xau"])
df = df.sort_values("date_xau")

# 2. Relevante Spalten in float konvertieren
for col in ["close_xau", "open_xau", "high_xau", "low_xau", "volume_xau"]:
    df[col] = pd.to_numeric(df[col], errors="coerce")

# Entferne Zeilen mit fehlendem Schlusskurs
df = df.dropna(subset=["close_xau"]).copy()

# 3. Technische Indikatoren berechnen
# RSI
rsi = RSIIndicator(close=df["close_xau"], window=14)
df["RSI"] = rsi.rsi()

# MACD
macd = MACD(close=df["close_xau"], window_slow=26, window_fast=12,
            ↪window_sign=9)
df["MACD"] = macd.macd()

# MA20
sma = SMAIndicator(close=df["close_xau"], window=20)
df["MA_20"] = sma.sma_indicator()

# Trendlinie (z.B. identisch mit MA_20)
df["Trendline"] = df["MA_20"]

# 4. Finale Spaltenauswahl und Umbenennung
df_out = df[["
```

```

        "date_xau", "open_xau", "high_xau", "low_xau", "close_xau",
        "RSI", "MACD", "MA_20", "Trendline", "volume_xau"
    ]]
    df_out.columns = [
        "timestamp", "open", "high", "low", "close",
        "RSI", "MACD", "MA_20", "Trendline", "volume"
    ]

    # 5. Runden und fehlende Werte als leere Felder
    df_out = df_out.round(5).fillna("")

    # 6. Falls Datei schon existiert → löschen
    output_path = "../dataset/output/xau_technical_clean.csv"
    if os.path.exists(output_path):
        os.remove(output_path)

    # 7. Speichern
    df_out.to_csv(output_path, index=False, sep=",")

```

2 Indikatoren für BTC berechnen und exportieren

```

[14]: import pandas as pd
import os
from ta.momentum import RSIIndicator
from ta.trend import MACD, SMAIndicator

# 1. Daten laden
df = pd.read_csv("../dataset/output/btc_clean.csv", parse_dates=["date_btc"])
df = df.sort_values("date_btc")

# 2. Relevante Spalten in float konvertieren
for col in ["close_btc", "open_btc", "high_btc", "low_btc", "volume_usd_btc"]:
    df[col] = pd.to_numeric(df[col], errors="coerce")

# Entferne Zeilen mit fehlendem Schlusskurs
df = df.dropna(subset=["close_btc"]).copy()

# 3. Technische Indikatoren berechnen
# RSI
rsi = RSIIndicator(close=df["close_btc"], window=14)
df["RSI"] = rsi.rsi()

# MACD
macd = MACD(close=df["close_btc"], window_slow=26, window_fast=12,
            ↪ window_sign=9)
df["MACD"] = macd.macd()

```

```

# MA20
sma = SMAIndicator(close=df["close_btc"], window=20)
df["MA_20"] = sma.sma_indicator()

# Trendlinie (z.B. identisch mit MA_20)
df["Trendline"] = df["MA_20"]

# 4. Finale Spaltenauswahl und Umbenennung
df_out = df[[
    "date_btc", "open_btc", "high_btc", "low_btc", "close_btc",
    "RSI", "MACD", "MA_20", "Trendline", "volume_usd_btc"
]]
df_out.columns = [
    "timestamp", "open", "high", "low", "close",
    "RSI", "MACD", "MA_20", "Trendline", "volume"
]

# 5. Runden und fehlende Werte als leere Felder
df_out = df_out.round(5).fillna("")

# 6. Falls Datei schon existiert → löschen
output_path = "../dataset/output/btc_technical_clean.csv"
if os.path.exists(output_path):
    os.remove(output_path)

# 7. Speichern
df_out.to_csv(output_path, index=False, sep=",")

```

3 Merge technische Indikatoren für BTC und XAU

```

[15]: import pandas as pd
import os

# 1. Dateien laden
btc_path = "../dataset/output/btc_technical_clean.csv"
xau_path = "../dataset/output/xau_technical_clean.csv"

btc = pd.read_csv(btc_path, parse_dates=["timestamp"])
xau = pd.read_csv(xau_path, parse_dates=["timestamp"])

# 2. Merge auf Basis des Zeitstempels
merged = pd.merge(btc, xau, on="timestamp", how="inner", suffixes=("_btc", "_xau"))

# 3. Sortierung und Bereinigung

```

```
merged = merged.sort_values("timestamp").reset_index(drop=True)

# 3.1 Entferne Zeilen mit fehlenden technischen Indikatoren (z. B. am Anfang)
merged = merged.dropna(subset=["RSI_btc", "MACD_btc", "RSI_xau", "MACD_xau"]).
↳ copy()

# 4. Falls Datei schon existiert → löschen
output_path = "../dataset/output/final_merged_technical_dataset.csv"
if os.path.exists(output_path):
    os.remove(output_path)

# 5. Speichern
merged.to_csv(output_path, index=False, sep=",")

print(" Merge abgeschlossen und gespeichert unter:", output_path)
```

Merge abgeschlossen und gespeichert unter:
../dataset/output/final_merged_technical_dataset.csv

4 Histogramme

4.0.1 für Vergleich zwischen Preise und technischen Indikatoren für BTC und XAU nebeneinander

```
[16]: import pandas as pd
import matplotlib.pyplot as plt

# 1. Daten laden
df = pd.read_csv("../dataset/output/final_merged_technical_dataset.csv",
↳ parse_dates=["timestamp"])

# Ziel:
# Vergleich der Verteilung technischer Indikatoren und Volumen für BTC und XAU -
direkt nebeneinander dargestellt

# 2. Paare definieren (BTC XAU)
paare = [
    ("close_btc", "close_xau"),
    ("MACD_btc", "MACD_xau"),
    ("RSI_btc", "RSI_xau"),
    ("MA_20_btc", "MA_20_xau"),
    ("volume_btc", "volume_xau") # NEU: Volumenvergleich
]

# 3. Plot
plt.figure(figsize=(14, 15)) # größer wegen zusätzlicher Zeile
```

```

for i, (btc_col, xau_col) in enumerate(paare, 1):
    # BTC links
    plt.subplot(len(paare), 2, (i - 1) * 2 + 1)
    df[btc_col].hist(bins=50, edgecolor="black")
    plt.title(f"BTC - {btc_col}")
    plt.xlabel(btc_col)
    plt.ylabel("Häufigkeit")

    # XAU rechts
    plt.subplot(len(paare), 2, (i - 1) * 2 + 2)
    df[xau_col].hist(bins=50, edgecolor="black", color="gold")
    plt.title(f"XAU - {xau_col}")
    plt.xlabel(xau_col)
    plt.ylabel("Häufigkeit")

plt.tight_layout()
plt.show()

```



5 Korrelation als Heatmap

5.0.1 berechnen und visualisieren die Korrelationen technischer Indikatoren innerhalb von Bitcoin sowie zwischen Bitcoin und Gold mithilfe zweier Heatmaps.

```
[27]: import seaborn as sns
import matplotlib.pyplot as plt

# Spaltennamen definieren
btc_vars = ["close_btc", "RSI_btc", "MACD_btc", "MA_20_btc", "volume_btc"]
```

```

gold_vars = ["close_xau", "RSI_xau", "MACD_xau", "MA_20_xau", "volume_xau"]

cols = btc_vars + gold_vars
label_map = {
    "close_btc": "BTC Preis",
    "RSI_btc": "BTC RSI",
    "MACD_btc": "BTC MACD",
    "MA_20_btc": "BTC MA20",
    "volume_btc": "BTC Volumen",
    "close_xau": "Gold Preis",
    "RSI_xau": "Gold RSI",
    "MACD_xau": "Gold MACD",
    "MA_20_xau": "Gold MA20",
    "volume_xau": "Gold Volumen"
}

# Korrelation berechnen und Spalten umbenennen
df_korr = df[cols].corr()
df_korr.columns = [label_map[col] for col in df_korr.columns]
df_korr.index = [label_map[idx] for idx in df_korr.index]

btc_vars_named = [label_map[v] for v in btc_vars]
gold_vars_named = [label_map[v] for v in gold_vars]

btc_corr = df_korr.loc[btc_vars_named, btc_vars_named]
gold_corr = df_korr.loc[gold_vars_named, gold_vars_named]
cross_corr = df_korr.loc[btc_vars_named, gold_vars_named]

# Layout: 2x2-Grid, untere rechte Zelle leer
fig, axes = plt.subplots(2, 2, figsize=(16, 12))

# Interne BTC
sns.heatmap(btc_corr, annot=True, cmap="coolwarm", fmt=".2f", center=0,
            ↪ax=axes[0, 0])
axes[0, 0].set_title("Interne Korrelationen - BTC")

# Interne Gold
sns.heatmap(gold_corr, annot=True, cmap="coolwarm", fmt=".2f", center=0,
            ↪ax=axes[0, 1])
axes[0, 1].set_title("Interne Korrelationen - Gold")

# BTC vs. Gold zentriert unten (linke untere Zelle)
sns.heatmap(cross_corr, annot=True, cmap="coolwarm", fmt=".2f", center=0,
            ↪ax=axes[1, 0])
axes[1, 0].set_title("Korrelation BTC vs. Gold")

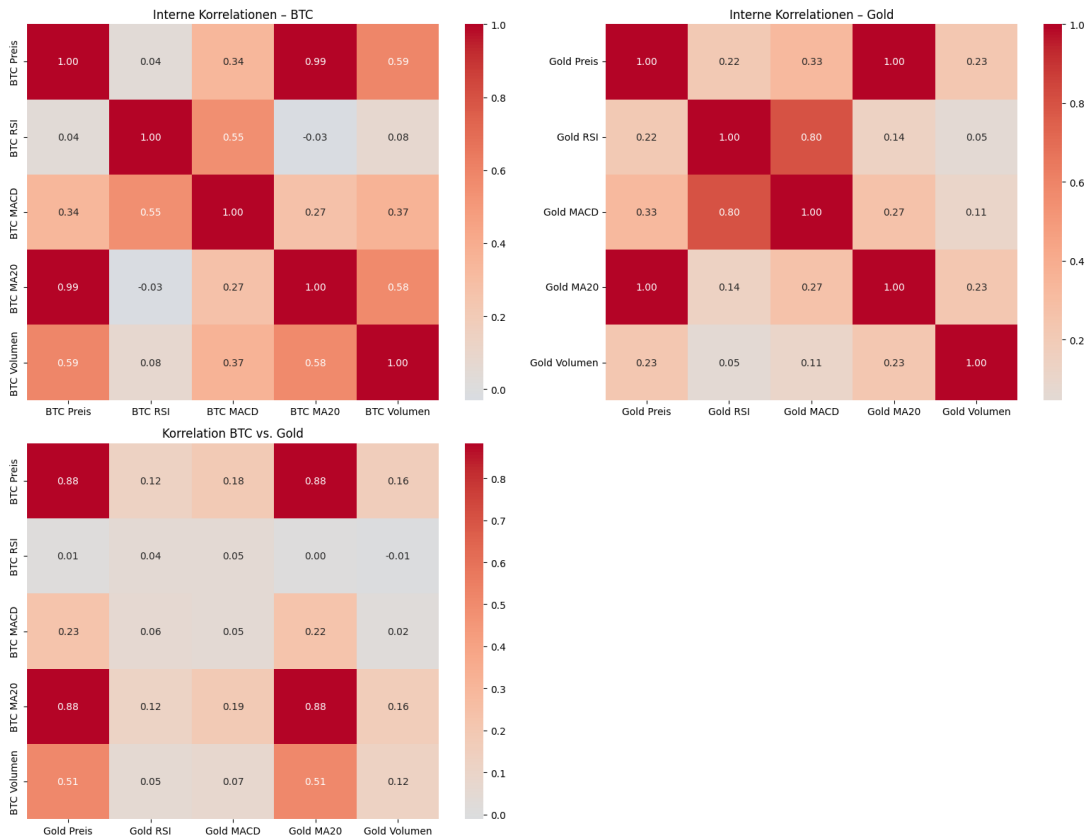
# Rechte untere Zelle leer lassen

```

```
axes[1, 1].axis("off")
```

```
plt.tight_layout()
```

```
plt.show()
```



6 Streudiagramme

6.0.1 Preis vs Indikatoren

```
[28]: fig, axes = plt.subplots(3, 2, figsize=(14, 14)) # 3 Zeilen, 2 Spalten
```

```
# BTC: RSI vs Preis
```

```
axes[0, 0].scatter(df["RSI_btc"], df["close_btc"], alpha=0.5,
```

```
↵c=df["volume_btc"], cmap="Blues")
```

```
axes[0, 0].set_title("BTC: RSI vs. Preis")
```

```
axes[0, 0].set_xlabel("RSI")
```

```
axes[0, 0].set_ylabel("Preis")
```

```
# Gold: RSI vs Preis
```



```

axes[0, 1].scatter(df["RSI_xau"], df["close_xau"], alpha=0.5,
    ↪c=df["volume_xau"], cmap="YlOrBr")
axes[0, 1].set_title("Gold: RSI vs. Preis")
axes[0, 1].set_xlabel("RSI")
axes[0, 1].set_ylabel("Preis")

# BTC: MACD vs Preis
axes[1, 0].scatter(df["MACD_btc"], df["close_btc"], alpha=0.5,
    ↪c=df["volume_btc"], cmap="Blues")
axes[1, 0].set_title("BTC: MACD vs. Preis")
axes[1, 0].set_xlabel("MACD")
axes[1, 0].set_ylabel("Preis")

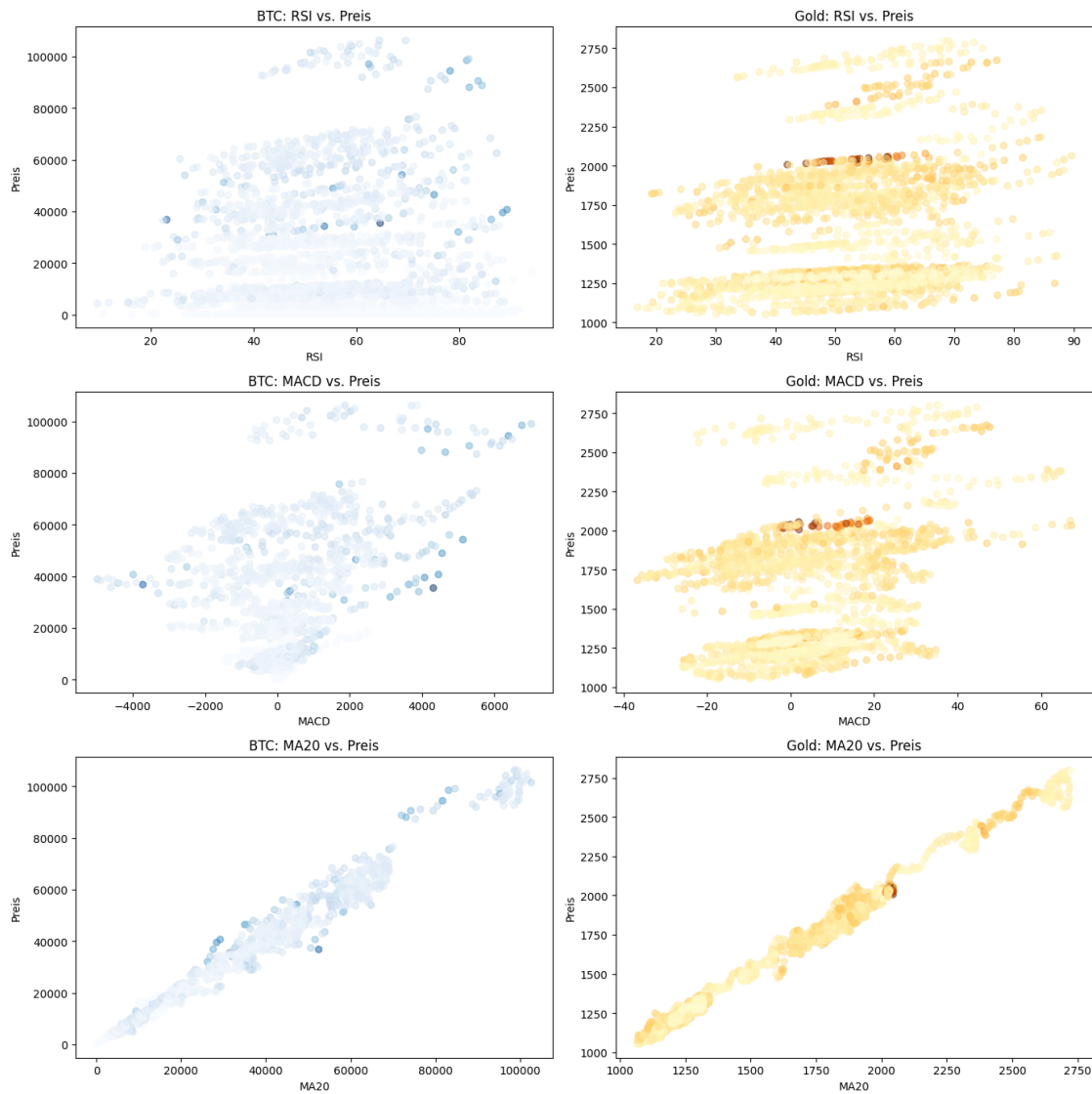
# Gold: MACD vs Preis
axes[1, 1].scatter(df["MACD_xau"], df["close_xau"], alpha=0.5,
    ↪c=df["volume_xau"], cmap="YlOrBr")
axes[1, 1].set_title("Gold: MACD vs. Preis")
axes[1, 1].set_xlabel("MACD")
axes[1, 1].set_ylabel("Preis")

# BTC: MA20 vs Preis
axes[2, 0].scatter(df["MA_20_btc"], df["close_btc"], alpha=0.5,
    ↪c=df["volume_btc"], cmap="Blues")
axes[2, 0].set_title("BTC: MA20 vs. Preis")
axes[2, 0].set_xlabel("MA20")
axes[2, 0].set_ylabel("Preis")

# Gold: MA20 vs Preis
axes[2, 1].scatter(df["MA_20_xau"], df["close_xau"], alpha=0.5,
    ↪c=df["volume_xau"], cmap="YlOrBr")
axes[2, 1].set_title("Gold: MA20 vs. Preis")
axes[2, 1].set_xlabel("MA20")
axes[2, 1].set_ylabel("Preis")

plt.tight_layout()
plt.show()

```



7 t-Test

7.0.1 Unterscheiden Sie die Indikatoren und Preise der beide Assets signifikant?

```
[45]: import numpy as np

# Ziel: Bootstrap-Test für Mittelwertunterschiede zwischen BTC- und XAU-Werten

# Liste der Paare: (btc_col, xau_col, beschriftung)
paare = [
    ("RSI_btc", "RSI_xau", "RSI"),
    ("MACD_btc", "MACD_xau", "MACD"),
```

```

    ("MA_20_btc", "MA_20_xau", "MA20"),
    ("volume_btc", "volume_xau", "Volumen"),
    ("close_btc", "close_xau", "Preis")
]

n_iter = 10000 # Anzahl Bootstrap-Iterationen

for btc_col, xau_col, label in paare:
    btc_values = df[btc_col].dropna().values
    xau_values = df[xau_col].dropna().values

    # Gleiche Länge erforderlich (optional: trimmen)
    min_len = min(len(btc_values), len(xau_values))
    btc_values = btc_values[:min_len]
    xau_values = xau_values[:min_len]

    diff_orig = np.mean(btc_values) - np.mean(xau_values)

    combined = np.concatenate([btc_values, xau_values])
    diffs = []

    for _ in range(n_iter):
        np.random.shuffle(combined)
        resample_btc = combined[:min_len]
        resample_xau = combined[min_len:]
        diff = np.mean(resample_btc) - np.mean(resample_xau)
        diffs.append(diff)

    p_value = np.mean(np.abs(diffs) >= np.abs(diff_orig))

    print(f"\n {label} - Bootstrapped p-Wert: {p_value:.4f}")
    if p_value < 0.05:
        print(f" Signifikanter Unterschied bei {label} zwischen BTC und Gold.")
    else:
        print(f" Kein signifikanter Unterschied bei {label}.")

```

RSI - Bootstrapped p-Wert: 0.0000

Signifikanter Unterschied bei RSI zwischen BTC und Gold.

MACD - Bootstrapped p-Wert: 0.0000

Signifikanter Unterschied bei MACD zwischen BTC und Gold.

MA20 - Bootstrapped p-Wert: 0.0000

Signifikanter Unterschied bei MA20 zwischen BTC und Gold.

Volumen - Bootstrapped p-Wert: 0.0000

Signifikanter Unterschied bei Volumen zwischen BTC und Gold.

Preis - Bootstrapped p-Wert: 0.0000
Signifikanter Unterschied bei Preis zwischen BTC und Gold.

8 ANOVA

8.0.1 Hängt der Bitcoin-Preis systematisch davon ab, wie stark Gold gehandelt wird?

```
[44]: import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# 1. Volumen-Gruppenbildung per Quantil
df["volume_xau_group"] = pd.qcut(df["volume_xau"], q=3, labels=["niedrig", "mittel", "hoch"])

# 2. BTC-Preise extrahieren je nach Gruppe
gruppen = {
    "niedrig": df[df["volume_xau_group"] == "niedrig"]["close_btc"].values,
    "mittel": df[df["volume_xau_group"] == "mittel"]["close_btc"].values,
    "hoch": df[df["volume_xau_group"] == "hoch"]["close_btc"].values
}

# 3. Originalmittelwert-Differenz berechnen
mittelwerte_orig = [np.mean(g) for g in gruppen.values()]
diff_orig = np.max(mittelwerte_orig) - np.min(mittelwerte_orig)

# 4. Bootstrap-Test durchführen
n_iter = 10000
diffs = []
all_values = np.concatenate(list(gruppen.values()))

for _ in range(n_iter):
    np.random.shuffle(all_values)
    resample = {
        "niedrig": all_values[:len(gruppen["niedrig"])],
        "mittel": all_values[len(gruppen["niedrig"]):
        ↪ len(gruppen["niedrig"])+len(gruppen["mittel"])],
        "hoch": all_values[-len(gruppen["hoch"]):]
    }
    means = [np.mean(v) for v in resample.values()]
    diff = np.max(means) - np.min(means)
    diffs.append(diff)

# 5. p-Wert berechnen
p_value = np.mean(np.array(diffs) >= diff_orig)
```

```

print(f"Bootstrapped p-Wert: {p_value:.4f}")

if p_value < 0.05:
    print(" Signifikanter Unterschied in BTC-Preisen je nach_
↪Gold-Volumen-Gruppe.")
else:
    print(" Kein signifikanter Unterschied in BTC-Preisen zwischen den Gruppen.
↪")

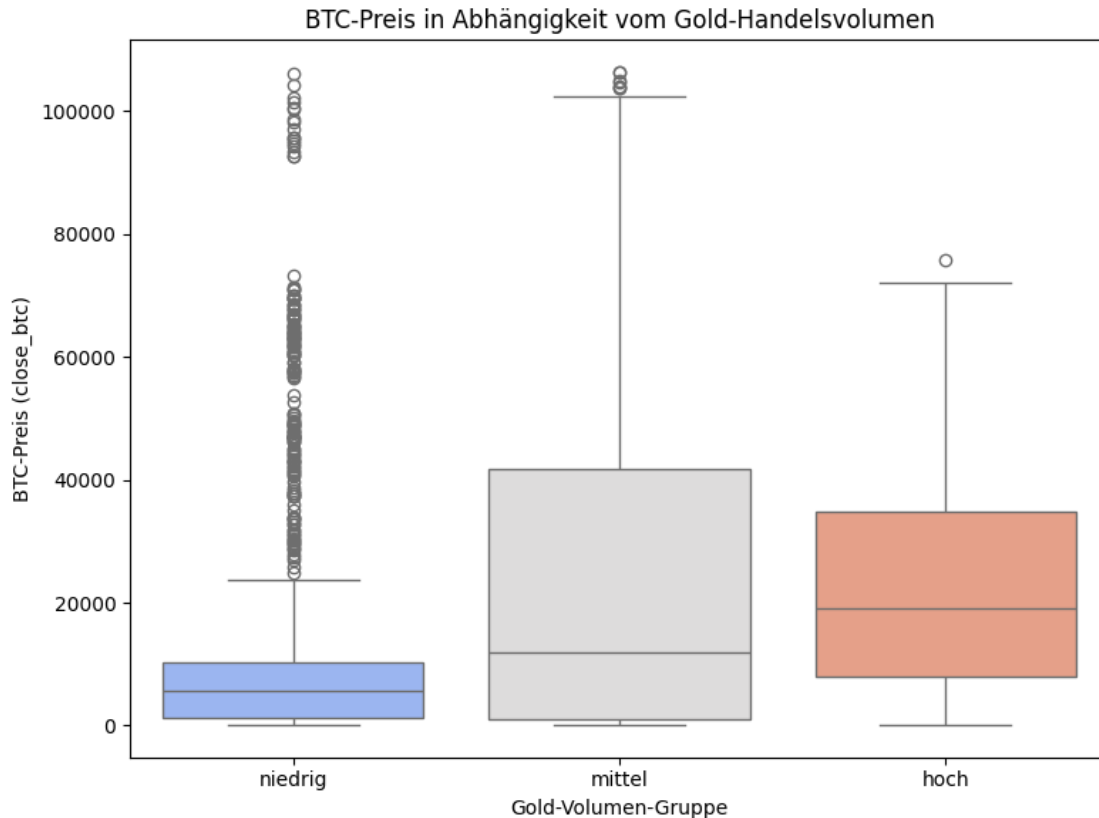
# 6. Visualisierung mit Boxplot (Future-safe)
plt.figure(figsize=(8, 6))
sns.boxplot(
    x="volume_xau_group",
    y="close_btc",
    hue="volume_xau_group",
    data=df,
    palette="coolwarm",
    legend=False
)

plt.title("BTC-Preis in Abhängigkeit vom Gold-Handelsvolumen")
plt.xlabel("Gold-Volumen-Gruppe")
plt.ylabel("BTC-Preis (close_btc)")
plt.tight_layout()
plt.show()

```

Bootstrapped p-Wert: 0.0000

Signifikanter Unterschied in BTC-Preisen je nach Gold-Volumen-Gruppe.



9 Modellierung: Multiples lineares Regressionsmodell (MLR) zur Vorhersage des Bitcoin-Schlusskurses (close_btc)

9.0.1 inklusive inferenzstatistischer Analyse der Regressionskoeffizienten (Konfidenzintervalle)

```
[4]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import statsmodels.api as sm
from sklearn.metrics import mean_squared_error, r2_score

# 1. Eingabefeatures & Zielvariable
df = pd.read_csv("../dataset/output/final_merged_technical_dataset.csv")
df = df.copy()

features = ["RSI_btc", "MACD_btc", "MA_20_btc", "RSI_xau", "MACD_xau", "
↪ "MA_20_xau"]
target = "close_btc"
```

```

# 2. Nur vollständige Zeilen verwenden
data = df[features + [target, "timestamp"]].dropna().copy()
data["timestamp"] = pd.to_datetime(data["timestamp"])

X = data[features]
y = data[target]

# 3. Zeitbasierter Split
train_size = int(len(data) * 0.8)
X_train, X_test = X.iloc[:train_size], X.iloc[train_size:]
y_train, y_test = y.iloc[:train_size], y.iloc[train_size:]
timestamps_test = data["timestamp"].iloc[train_size:]

# 4. Modell trainieren mit statsmodels (inkl. Konstante)
X_train_const = sm.add_constant(X_train) # Interzept (Achsenabschnitt)
↳ hinzufügen
model = sm.OLS(y_train, X_train_const).fit()

# 5. Zusammenfassung inkl. Konfidenzintervalle
description = model.summary()
print(description)

# 6. Vorhersage & Bewertung
X_test_const = sm.add_constant(X_test)
y_pred = model.predict(X_test_const)

r2 = r2_score(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))

print(f"\n R2 (Bestimmtheitsmaß): {r2:.4f}")
print(f" RMSE (Root Mean Squared Error): {rmse:.2f}")

# 7. Visualisierung - zwei Plots nebeneinander
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# A: Scatterplot Vorhersage vs. Realität
axes[0].scatter(y_test, y_pred, alpha=0.6, edgecolors='k')
axes[0].plot([y.min(), y.max()], [y.min(), y.max()], 'r--')
axes[0].set_xlabel("Tatsächlicher BTC-Preis")
axes[0].set_ylabel("Vorhergesagter BTC-Preis")
axes[0].set_title("Vorhersage vs. Realität (Scatter)")

# B: Zeitverlauf - Realität vs. Vorhersage
axes[1].plot(timestamps_test, y_test.values, label="Tatsächlich", color="black")
axes[1].plot(timestamps_test, y_pred, label="Vorhersage", color="orange",
↳ linestyle="--")
axes[1].set_title("BTC-Preis über Zeit - Realität vs. Vorhersage")

```

```

axes[1].set_xlabel("Zeit")
axes[1].set_ylabel("BTC-Preis")
axes[1].legend()

plt.tight_layout()
plt.show()

```

OLS Regression Results

```

=====
Dep. Variable:          close_btc      R-squared:                0.994
Model:                  OLS           Adj. R-squared:           0.994
Method:                 Least Squares  F-statistic:              5.615e+04
Date:                  Sun, 08 Jun 2025  Prob (F-statistic):        0.00
Time:                  07:06:03        Log-Likelihood:           -17817.
No. Observations:      2080           AIC:                    3.565e+04
Df Residuals:          2073           BIC:                    3.569e+04
Df Model:               6
Covariance Type:       nonrobust
=====

```

	coef	std err	t	P> t	[0.025	0.975]
const	-3567.4957	324.612	-10.990	0.000	-4204.096	-2930.895
RSI_btc	35.8525	2.379	15.073	0.000	31.188	40.517
MACD_btc	1.3073	0.034	38.480	0.000	1.241	1.374
MA_20_btc	0.9797	0.003	350.495	0.000	0.974	0.985
RSI_xau	6.2429	3.855	1.620	0.105	-1.316	13.802
MACD_xau	-10.5090	3.498	-3.004	0.003	-17.370	-3.648
MA_20_xau	1.1220	0.164	6.831	0.000	0.800	1.444

```

=====
Omnibus:                341.143      Durbin-Watson:           0.373
Prob(Omnibus):          0.000        Jarque-Bera (JB):        6118.535
Skew:                   0.108        Prob(JB):                0.00
Kurtosis:               11.400        Cond. No.                2.44e+05
=====

```

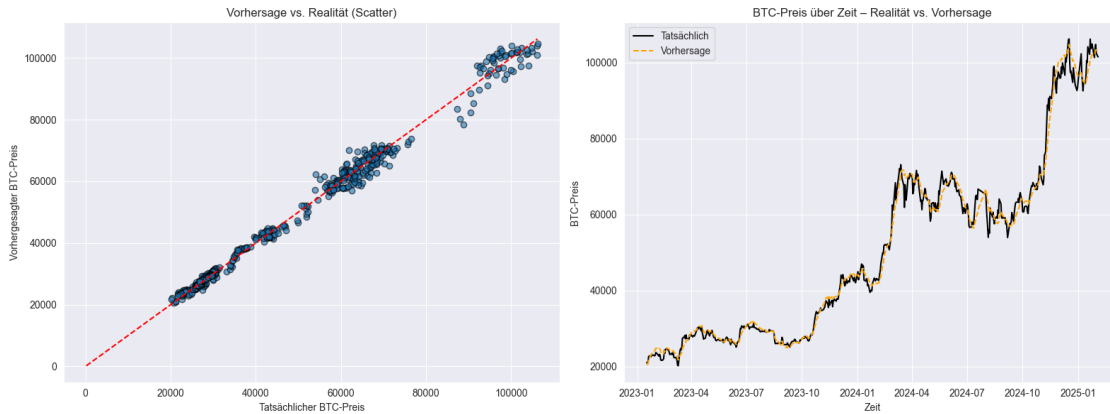
Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 2.44e+05. This might indicate that there are strong multicollinearity or other numerical problems.

R² (Bestimmtheitsmaß): 0.9918

RMSE (Root Mean Squared Error): 2133.65



10 Modelling: Naive Bayes & LDA

10.0.1 Verwende Naive Bayes oder LDA zur Vorhersage der Marktpphase (bull / bear / sideways) basierend auf technischen Indikatoren für BTC und XAU.

```
[43]: import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.metrics import classification_report

# Daten laden
df = pd.read_csv("../dataset/output/final_merged_technical_dataset.csv",
                 parse_dates=["timestamp"])

# Zielvariable: Marktpphase anhand Kursveränderung
def classify_phase(change_pct, threshold_pct=2.0):
    if change_pct > threshold_pct:
        return "bull"
    elif change_pct < -threshold_pct:
        return "bear"
    else:
        return "sideways"

df["price_change_pct_btc"] = df["close_btc"].pct_change(5) * 100
df["market_phase_btc"] = df["price_change_pct_btc"].apply(classify_phase)

# Features definieren (BTC & XAU Indikatoren)
features = [
    "RSI_btc", "MACD_btc", "MA_20_btc", "Trendline_btc",
    "RSI_xau", "MACD_xau", "MA_20_xau", "Trendline_xau"
]
```

```

df_model = df.dropna(subset=features + ["market_phase_btc"])
X = df_model[features]
y = df_model["market_phase_btc"]

# Train-Test-Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
    random_state=42)

# 1 Naiver Bayes
nb_model = GaussianNB()
nb_model.fit(X_train, y_train)
y_pred_nb = nb_model.predict(X_test)
print(" Naiver Bayes Klassifikation:\n", classification_report(y_test,
    y_pred_nb))

# 2 LDA
lda_model = LinearDiscriminantAnalysis()
lda_model.fit(X_train, y_train)
y_pred_lda = lda_model.predict(X_test)
print(" LDA Klassifikation:\n", classification_report(y_test, y_pred_lda))

```

Naiver Bayes Klassifikation:

	precision	recall	f1-score	support
bear	0.55	0.66	0.60	152
bull	0.75	0.76	0.76	229
sideways	0.39	0.29	0.33	140
accuracy			0.61	521
macro avg	0.56	0.57	0.56	521
weighted avg	0.60	0.61	0.60	521

LDA Klassifikation:

	precision	recall	f1-score	support
bear	0.57	0.75	0.65	152
bull	0.77	0.79	0.78	229
sideways	0.47	0.29	0.36	140
accuracy			0.64	521
macro avg	0.60	0.61	0.60	521
weighted avg	0.63	0.64	0.63	521